

Spotify Music Graph in Neo4j

Objective

In this project, you will build a Spotify-style music graph using Neo4j and a Kaggle dataset.

You will:

- Clean a real dataset using Python + pandas.
- Generate CSV files for nodes and relationships.
- Import data into Neo4j with simple **LOAD CSV** + **MERGE** Cypher.
- Work with four node labels: Track, Artist, Album, Genre.
- **Write Cypher queries that use filters, ORDER BY, LIMIT, and other CRUD operations.**
- Measure query execution time for each query.

Dataset:

<https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset?resource=download&select=dataset.csv>

Use a Neo4j database name **spotifydb**.

Part 1: Data Cleaning with Python and Pandas (10%)

- Download **dataset.csv** from Kaggle and place it in your working directory.
- In Python, read and clean the data with a pandas DataFrame:

```
import pandas as pd
```

```
df = pd.read_csv("dataset.csv")
```

```
critical_cols = [  
    "track_id",  
    "track_name",  
    "artists",  
    "album_name",  
    "track_genre",  
    "popularity",  
    "danceability",  
    "energy",  
    "acousticness",  
    "valence",  
    "tempo"
```

```

]

# Drop rows with missing values in critical columns
df = df.dropna(subset=critical_cols)

print(df.shape) # to inspect number of rows
print(df.head()) # preview first rows

```

After cleaning, make sure you have **at least 20,000 rows**.

There is **no maximum**; you may keep all rows that remain after cleaning.

You will use this cleaned DataFrame to generate CSV files in Part 2.

It's up to you to use pandas or other techniques. These parts are to help you in case you do it in pandas.

Part 2: Generating CSV Files for Neo4j (20%)

From the cleaned DataFrame `df`, create separate CSV files for nodes and relationships.

2.1: tracks.csv

Columns:

- track_id
- track_name
- popularity
- danceability
- valence
- acousticness
- album_name
- track_genre
- artists (semicolon-separated artist names)

Example:

```

df_tracks = df[[
    "track_id",
    "track_name",
    "popularity",
    "danceability",
    "valence",
    "acousticness",

```

```

    "album_name",
    "track_genre",
    "artists"
]).drop_duplicates(subset=["track_id"])

df_tracks.to_csv("tracks.csv", index=False)

```

2.2: artists.csv

Unique artist names.

```

df_tracks["artist_list"] = df_tracks["artists"].astype(str).str.split(",")

unique_artists = set(
    artist.strip()
    for sublist in df_tracks["artist_list"]
    for artist in sublist
)

df_artists = pd.DataFrame(unique_artists, columns=["name"])
df_artists.to_csv("artists.csv", index=False)

```

2.3: albums.csv

```

df_albums = df_tracks[["album_name"]].drop_duplicates()
df_albums.rename(columns={"album_name": "name"}, inplace=True)
df_albums.to_csv("albums.csv", index=False)

```

2.4: genres.csv

```

df_genres = df_tracks[["track_genre"]].drop_duplicates()
df_genres.rename(columns={"track_genre": "name"}, inplace=True)
df_genres.to_csv("genres.csv", index=False)

```

2.5a: rel_performed_by.csv – (Track) – [:PERFORMED_BY] -> (Artist)

```

rows = []
for _, row in df_tracks.iterrows():
    track_id = row["track_id"]
    for artist in row["artist_list"]:
        rows.append({
            "track_id": track_id,
            "artist_name": artist.strip()
        })

df_performed_by = pd.DataFrame(rows)
df_performed_by.to_csv("rel_performed_by.csv", index=False)

```

Columns: track_id, artist_name.

2.5b: rel_belongs_to.csv – (Track) – [:BELONGS_TO] -> (Album)

```

df_belongs_to = df_tracks[["track_id", "album_name"]].drop_duplicates()
df_belongs_to.to_csv("rel_belongs_to.csv", index=False)

```

Columns: track_id, album_name.

2.5c: rel_has_genre.csv – (Track) – [:HAS_GENRE] -> (Genre)

```

df_has_genre = df_tracks[["track_id", "track_genre"]].drop_duplicates()
df_has_genre.to_csv("rel_has_genre.csv", index=False)

```

Columns: track_id, track_genre.

You will submit the Python script that produces these CSV files.

Part 3: Importing to Neo4j (30%)

Copy all CSVs into Neo4j's `import/` directory:

- tracks.csv
- artists.csv
- albums.csv
- genres.csv
- rel_performed_by.csv

- `rel_belongs_to.csv`
- `rel_has_genre.csv`

You must use exactly these labels: **:Track**, **:Artist**, **:Album**, **:Genre** and relationships: **:PERFORMED_BY**, **:BELONGS_TO**, **:HAS_GENRE**

3.1 Create Track nodes

For example:

```
LOAD CSV WITH HEADERS FROM 'file:///tracks.csv' AS row
```

```
MERGE (t:Track {track_id: row.track_id})
SET t.track_name = row.track_name,
    t.popularity = toInteger(row.popularity),
    t.danceability = toFloat(row.danceability),
    t.valence = toFloat(row.valence),
    t.acousticness = toFloat(row.acousticness);
```

3.2 Create Artist, Album, Genre nodes

// Artists

```
LOAD CSV WITH HEADERS FROM 'file:///artists.csv' AS row
```

```
....
```

```
....
```

// Albums

```
LOAD CSV WITH HEADERS FROM 'file:///albums.csv' AS row
```

```
....
```

```
....
```

// Genres

```
LOAD CSV WITH HEADERS FROM 'file:///genres.csv' AS row
```

```
....
```

```
....
```

Fill the body of the commands with the correct data.

3.3 Create relationships (three types)

```
// PERFORMED_BY
LOAD CSV WITH HEADERS FROM 'file:///rel_performed_by.csv' AS row
....
....
// BELONGS_TO
LOAD CSV WITH HEADERS FROM 'file:///rel_belongs_to.csv' AS row
....
....

// HAS_GENRE
LOAD CSV WITH HEADERS FROM 'file:///rel_has_genre.csv' AS row
....
....
```

Fill the body of the commands with the correct data.

Part 4: Cypher Query Tasks (40%)

For each task:

- Write the **Cypher query**.
- Make sure it runs correctly on your data.
- Measure and report **execution time** (see Execution Time section).
- Screenshots (see deliverables part)

Task 1: Highly Popular Tracks

List the **top 20** tracks with **popularity > 80**.

- Return: **track_name**, **popularity**.
 - Sort by **popularity descending** and limit to 20 rows.
-

Task 2: Tracks with “Love” or “Dream” in the Title

Find up to **50** tracks whose title contains “love” or “dream”, case-insensitive.

- Return: **track_name**, **popularity**.
 - Sort by **popularity descending**.
 - Limit the output to 50 rows.
-

Task 3: Tracks by a Specific Artist

Given an artist name (e.g., "Taylor Swift"), show all tracks performed by that artist.

- Use the `PERFORMED_BY` relationship.
 - Return: `track_name`, `popularity`.
 - Sort by `popularity` **descending**.
-

Task 4: Upbeat & Happy Tracks

Find tracks that satisfy:

- `danceability` above a threshold you choose (e.g., `> 0.7`),
 - `valence` above a threshold you choose (e.g., `> 0.7`),
 - `popularity > 60`.
 - Return: `track_name`, `popularity`, `danceability`, `valence`.
 - Sort by `popularity` **descending**.
 - Limit the result to **50** tracks.
 - In your explanation, mention the thresholds you used.
-

Task 5: Update Artist Name

Rename artist "Jason Mraz" to "Jason M."

- Update the `Artist` node's `name` property.
 - Then query for `Artist` nodes with `name = "Jason M."` to confirm.
 - Sorting is optional here.
-

Task 6: Average Track Popularity by Genre

Compute the **average popularity** of tracks for each genre.

- Use `HAS_GENRE` relationships.
 - Return: `genre name`, `avg_popularity`.
 - Sort by `avg_popularity` **descending**.
-

Task 7: Genre-wise Track Counts

Count how many tracks belong to each genre.

- Use `HAS_GENRE`.
- Return: `genre name`, `track_count`.

- Sort by `track_count` **descending**.
 - Limit to the **top 10** genres.
-

Part 5: Deliverables

You will submit a **single ZIP** file:

group#_Option3_Project_11.zip

The ZIP must contain:

Solutions.txt

For **each task**, include:

- Task number and title.
- Your Neo4j query
- Execution time:
 - 10 run times (you can show them as a small list).
 - The **average execution time** in seconds.

Python Scripts

Include:

The Python script(s) used to clean the data and generate the CSV files:

- tracks.csv
- artists.csv
- albums.csv
- genres.csv
- rel_performed_by.csv
- rel_belongs_to.csv
- rel_has_genre.csv

If you used more than one script, that's fine – just include them all.

Screenshots

Create a folder:

Screenshots/

For each task:

- The query in Neo4j Browser/Desktop.
 - The first part of the result (you can use `LIMIT`).
 - Where you printed the average execution time (if visible).
-

Execution Time Measurement

For every task:

- Run the same query **10 times**.
- Measure the time for each run.
- Compute the **average execution time**.

You may use Python with the Neo4j driver and the `time` module. Example pattern:

Python Timing Pattern

Example pattern (you can adapt):

```
import time
from neo4j import GraphDatabase

driver = GraphDatabase.driver("bolt://localhost:7687", auth=("neo4j", "password"))

query = """
MATCH (t:Track)
WHERE t.popularity > 80
RETURN t.track_name, t.popularity
ORDER BY t.popularity DESC
LIMIT 20
"""

with driver.session() as session:
    total = 0.0
    times = []
    for i in range(10):
        start = time.time()
        result = session.run(query).list()
        end = time.time()
```

```
run_time = end - start
times.append(run_time)
print(f"Run {i+1}: {run_time:.6f} seconds")
total += run_time
```

```
average_time = total / 10
print("Average execution time (s):", average_time)
```